

Formal Verification: Safety Properties and Model Checking

Formal Verification: Safety Properties and Model Checking

This section establishes safety properties
(`sec:safety-properties`), proves invariant preservation lemmas
(`sec:invariant-lemmas`), demonstrates liveness guarantees
(`sec:liveness-properties`), derives complexity bounds
(`sec:complexity-bounds`), and presents model checking
verification (`sec:model-checking`).

Safety Properties I

Safety Properties II

Belief Integrity

Theorem (Belief Injection Resistance)

Under CIF with firewall detection rate r_f and sandboxing verification rate r_s :

$$P(\mathcal{A}_{BI} \text{ succeeds}) \leq (1 - r_f) \cdot (1 - r_s) \quad (1)$$

Proof.

We prove this theorem by analyzing the sequential defense mechanism and applying probability theory for independent events.

Setup: Let ϕ_{adv} be an adversarial belief that the attacker attempts to inject into agent a_i 's verified belief set $\mathcal{B}_{verified}$.

Defense Model: The CIF implements two sequential defenses:

1. **Cognitive Firewall $\mathcal{F}$:** Classifies incoming messages as ACCEPT, QUARANTINE, or REJECT with detection rate r_f
2. **Belief Sandbox:** Verifies quarantined beliefs before promotion with verification rate r_s

Invariant Preservation Lemmas I

Lemma (Belief Consistency Preservation)

*If $\mathcal{B}_i^t$ is consistent and the belief update follows Rule B-DIRECT or B-DELEGATED (*sec:belief-update-rules*), then $\mathcal{B}_i^{t+1}$ is consistent.*

Invariant Preservation Lemmas II

Proof.

Let $\text{Consistent}(\mathcal{B})$ denote that no high-confidence contradictions exist:

$$\text{Consistent}(\mathcal{B}) \iff \nexists \phi, \psi : \mathcal{B}(\phi) > \tau \wedge \mathcal{B}(\psi) > \tau \wedge (\phi \wedge \psi \vdash \perp) \quad (12)$$

Case 1: Rule B-DIRECT applies.

The update adds evidence for proposition ϕ :

$$\mathcal{B}_i^{t+1}(\phi) = \text{BayesUpdate}(\mathcal{B}_i^t(\phi), c \cdot \mathcal{T}_{i \rightarrow s}) \quad (13)$$

If the update would create contradiction (i.e., both $\mathcal{B}^{t+1}(\phi) > \tau$ and $\mathcal{B}^{t+1}(\psi) > \tau$ for contradictory ϕ, ψ), the sandbox consistency check in Rule S-PROMOTE rejects promotion:

$$V(\pi) = 1 \wedge \text{Consistent}(\mathcal{B}_{\text{verified}} \cup \{\phi\}) \wedge |\text{Corroborate}(\phi)| \geq \kappa \quad (14)$$

Therefore, only consistent updates reach $\mathcal{B}_{\text{verified}}$.

Case 2: Rule B-DELEGATED applies.

Same argument, with additional trust decay ensuring lower

Liveness Properties I

Non-Blocking

Theorem (Firewall Liveness)

CIF firewall preserves liveness for legitimate inputs:

$$\forall m \in \mathcal{M}_{\text{legitimate}} : P(\mathcal{F}(m) = \text{ACCEPT}) \geq 1 - \epsilon_{fp} \quad (23)$$

Proof.

By firewall design, false positive rate is bounded by ϵ_{fp} . Legitimate messages are rejected only on false positive, establishing the bound. □

Liveness Properties II

Progress Guarantee

Theorem (Byzantine Consensus Termination)

With $n \geq 3f + 1$ agents and at most f Byzantine:

$$P(\text{consensus reached in } O(f + 1) \text{ rounds}) = 1 \quad (24)$$

Proof.

Standard Byzantine agreement result (Lamport et al., 1982). With honest majority $n \geq 3f + 1$, the PBFT protocol guarantees termination in $f + 1$ rounds. □

Complexity Bounds I

Complexity Bounds II

Space Complexity

Table 1: Per-component space complexity.

Component	Space	Notes
Belief state	$O(\Phi)$	Propositions tracked
Provenance	$O(\Phi \cdot d)$	$d = \text{max chain depth}$
Trust matrix	$O(n^2)$	Pairwise trust
Tripwires	$O(k)$	$k = \text{canary count}$
History	$O(w)$	Window size

Theorem (Total Space Bound)

The total space complexity of CIF for n agents with $|\Phi|$ propositions, provenance depth d , k tripwires, and window size w is:

$$S_{total} = O(n \cdot (|\Phi| \cdot d + k + w) + n^2) \quad (25)$$

Proof.

Per agent:

- ▶ Belief state: $|\Phi|$ propositions with confidence values = $O(|\Phi|)$
- ▶ Provenance: Each belief has chain of depth at most $d = O(|\Phi| \cdot d)$
- ▶ Tripwires: k canary beliefs = $O(k)$

Formal Model Checking I

State Space Definition

Definition (System State)

The complete system state is the tuple:

$$s = (\sigma_1, \dots, \sigma_n, \mathcal{S}, \mathcal{T}) \quad (29)$$

where σ_i is agent i 's cognitive state, $\mathcal{S}$ is shared state, and $\mathcal{T}$ is the trust matrix.

Formal Model Checking II

Temporal Properties

We verify the following CTL (Computation Tree Logic) properties:
[Safety: Consensus Integrity]

$$AG(\neg \text{compromised}(\mathcal{B}_{\text{consensus}})) \quad (30)$$

“Always globally, consensus beliefs are not compromised.”
[Liveness: Request-Response]

$$AG(\text{request} \Rightarrow AF(\text{response})) \quad (31)$$

“Every request eventually gets a response.”
[Fairness: Tripwire Checking]

$$AG(AF(\text{tripwire_checked})) \quad (32)$$

“Tripwires are checked infinitely often.”

Formal Model Checking III

Model Checking Results

The following table summarizes the expected state space exploration for each property based on formal analysis of the CIF specification. These values represent theoretical bounds derived from the state space definition

(def:system-state-verification) and complexity analysis (sec:complexity-bounds). Actual model checking execution using NuSMV, SPIN, and TLA+ tooling is presented in Part 2 of this series, along with full implementation configurations.

Table 3: Model checking verification results. “Verified” here denotes a theoretical state-space bound established in this part; executable NuSMV/SPIN/TLA+ runs are deferred to Part 2 §04 (S04). No model checker was executed in this part.

Property	Status	Reference
Belief integrity	theoretical bound; executable run in Part 2	§S04 thm:belief-injection
Trust bounded	theoretical bound; executable run in Part 2	§S04 thm:trust-bound
No deadlock	theoretical bound; executable run in Part 2	§S04 thm:firewall-liveness
Eventual detection	theoretical bound; executable run in Part 2	§S04 thm:progressive-detection

Note: Model checking tool configurations (NuSMV, SPIN, TLA+) and verification parameters are provided in Part 2: Computational Validation, which presents the executable